

Técnicas de Mecanistic Interpretability para Modelos de Lenguaje Basados en Transformers

Informe Técnico Detallado

Julio 2026

Índice general

1. Introducción	5
1.1. Motivación	5
1.2. Estructura de este informe	5
1.3. Arquitectura de referencia	6
2. Logit Lens	8
2.1. Problema que resuelve	8
2.2. Fundamento matemático	8
2.2.1. La idea central	8
2.2.2. Interpretación como refinamiento iterativo	8
2.2.3. Ejemplo concreto	9
2.3. Variante: Tuned Lens	9
2.3.1. Definición formal	9
2.3.2. Entrenamiento	9
2.3.3. Mejora cualitativa	10
2.4. Limitaciones	10
3. Probing	11
3.1. Problema que resuelve	11
3.2. Fundamento matemático	11
3.2.1. Definición formal	11
3.2.2. Métrica de “representación”	11
3.2.3. Hipótesis de linealidad	12
3.3. Sparse Probing	12
3.3.1. Definición	12
3.3.2. Métodos de solución	12
3.3.3. Hallazgos empíricos	12
3.4. Ejemplo práctico	13
3.5. Limitaciones	13

4. Sparse Autoencoders (SAE)	14
4.1. Problema que resuelve	14
4.2. Fundamento matemático	14
4.2.1. El problema de la superposición	14
4.2.2. Arquitectura del SAE	14
4.2.3. Función de pérdida	15
4.2.4. Variante: TopK SAE	15
4.2.5. Entrenamiento	15
4.2.6. Features muertas	15
4.3. Ejemplo práctico	16
4.4. Limitaciones	16
5. Automated Feature Explanation	17
5.1. Problema que resuelve	17
5.2. El pipeline de tres fases	17
5.2.1. Fase 1: Generación de explicación	17
5.2.2. Fase 2: Simulación	17
5.2.3. Fase 3: Puntuación	18
5.3. Resultados empíricos	18
5.4. Ejemplo práctico	18
5.5. Limitaciones	19
6. Mean Ablation	20
6.1. Problema que resuelve	20
6.2. Fundamento matemático	20
6.2.1. Definición	20
6.2.2. Propiedades	20
6.2.3. Aplicación: el circuito IOI	21
6.3. Ejemplo práctico	21
6.4. Limitaciones	21
7. Path Patching	22
7.1. Problema que resuelve	22
7.2. Fundamento matemático	22
7.2.1. Configuración experimental	22
7.2.2. El procedimiento de patching	22
7.2.3. Tipos de Path Patching	23

7.3. Ejemplo práctico: el circuito IOI	23
7.4. Limitaciones	24
8. Random Ablation	25
8.1. Problema que resuelve	25
8.2. Fundamento matemático	25
8.2.1. Definición	25
8.2.2. Propiedades deseables	25
8.2.3. Promedio sobre múltiples ablaciones	26
8.3. Comparación: Mean Ablation vs. Random Ablation	26
8.4. Ejemplo práctico	26
8.5. Limitaciones	26
9. Causal Scrubbing	27
9.1. Problema que resuelve	27
9.2. Fundamento matemático	27
9.2.1. La hipótesis como grafo	27
9.2.2. El procedimiento de scrubbing	27
9.2.3. Resampling condicional	28
9.3. Métrica: <i>Recovered Log-Probability</i>	28
9.4. Ejemplo práctico	28
9.5. Limitaciones	29
10. Activation Patching	30
10.1. Problema que resuelve	30
10.2. Fundamento matemático	30
10.2.1. Configuración experimental	30
10.2.2. Procedimiento paso a paso	30
10.2.3. Efecto Indirecto Total	31
10.3. Aplicación: ROME (Model Editor)	31
10.4. Ejemplo práctico	31
10.5. Limitaciones	32
11. ACDC (Automatic Circuit DisCOVERY)	33
11.1. Problema que resuelve	33
11.2. Fundamento matemático	33
11.2.1. El algoritmo	33
11.2.2. Métrica de efecto	34

11.2.3. El umbral τ	34
11.3. Resultados empíricos	34
11.4. Ejemplo práctico	34
11.5. Limitaciones	35
12. Attribution Patching	36
12.1. Problema que resuelve	36
12.2. Fundamento matemático	36
12.2.1. La aproximación de Taylor	36
12.2.2. Cómputo eficiente	36
12.2.3. Interpretación intuitiva	37
12.3. Fallos conocidos (y AtP*)	37
12.3.1. Fallo por saturación de softmax	37
12.3.2. Fallo por cancelación frágil	37
12.4. Ejemplo práctico	37
12.5. Limitaciones	38
13. Edge Attribution Patching (EAP)	39
13.1. Problema que resuelve	39
13.2. Fundamento matemático	39
13.2.1. De nodos a aristas	39
13.2.2. El algoritmo EAP	39
13.2.3. Comparación EAP vs. ACDC	40
13.3. Variante: EAP con integradores	40
13.4. Ejemplo práctico	40
13.5. Limitaciones	41
14. Conclusiones	42
14.1. Tabla comparativa	42
14.2. Recomendaciones prácticas	43
14.3. Direcciones futuras	44

Capítulo 1

Introducción

La **Mecanistic Interpretability** (MI) es una rama de la inteligencia artificial que busca comprender el funcionamiento interno de las redes neuronales mediante un proceso de *ingeniería inversa*. A diferencia de enfoques puramente conductistas que solo observan entradas y salidas, la MI aspira a descomponer el comportamiento de un modelo entrenado en componentes interpretables: circuitos, características (features), neuronas y mecanismos causales.

Los modelos de lenguaje basados en transformers [1] han alcanzado capacidades notables en una amplia variedad de tareas, desde generación de texto hasta razonamiento matemático. Sin embargo, su complejidad interna—miles de millones de parámetros, docenas de capas, múltiples cabezas de atención—los convierte en auténticas *cajas negras*. La Mecanistic Interpretability proporciona un conjunto de herramientas analíticas para abrir esas cajas negras y entender qué hace cada componente y por qué.

1.1. Motivación

Comprender el funcionamiento interno de los modelos de lenguaje no es solo un ejercicio académico. La MI tiene implicaciones directas para:

- **Seguridad y alineamiento:** verificar que el modelo no utiliza atajos espurios, sesgos indeseados o comportamientos engañosos.
- **Depuración y mejora:** identificar fallos sistemáticos y repararlos mediante edición localizada de parámetros.
- **Transferencia y generalización:** entender qué mecanismos se reutilizan entre tareas, facilitando el diseño de modelos más eficientes.
- **IA explicable:** proporcionar explicaciones causales—no solo correlacionales—de las predicciones del modelo.

1.2. Estructura de este informe

Este informe presenta doce técnicas fundamentales de Mecanistic Interpretability, organizadas en capítulos independientes. Cada capítulo sigue una estructura pedagógica uniforme:

1. **Problema que resuelve:** contexto y motivación específica.

2. **Fundamento matemático:** definiciones formales y notación.
3. **Detalles de implementación:** cómo se aplica la técnica en la práctica.
4. **Ejemplo ilustrativo:** un caso concreto y sencillo que muestra el funcionamiento.
5. **Limitaciones y consideraciones:** advertencias sobre cuándo y cómo usar la técnica.

Las técnicas se presentan en el siguiente orden:

1. **Logit Lens** (Capítulo 2) — proyección de estados intermedios al espacio de vocabulario.
2. **Probing** (Capítulo 3) — clasificadores lineales sobre activaciones.
3. **SAE / Sparse Autoencoders** (Capítulo 4) — descomposición de activaciones en features monosemánticas.
4. **Automated Feature Explanation** (Capítulo 5) — etiquetado automático de features con LLMs.
5. **Mean Ablation** (Capítulo 6) — reemplazo de activaciones por su media.
6. **Path Patching** (Capítulo 7) — aislamiento causal de rutas específicas.
7. **Random Ablation** (Capítulo 8) — reemplazo preservando la distribución de activaciones.
8. **Causal Scrubbing** (Capítulo 9) — verificación formal de hipótesis interpretativas.
9. **Activation Patching** (Capítulo 10) — intervención causal para localizar componentes.
10. **ACDC** (Capítulo 11) — descubrimiento automático de circuitos.
11. **Attribution Patching** (Capítulo 12) — aproximación eficiente del Activation Patching.
12. **Edge Attribution Patching** (Capítulo 13) — atribución de aristas a escala industrial.

El informe concluye con una tabla comparativa que resume las fortalezas, debilidades y casos de uso de cada técnica, junto con recomendaciones prácticas para investigadores que se inician en este campo.

1.3. Arquitectura de referencia

Para facilitar la exposición, asumimos un transformer estándar como el de GPT-2 [2]. Un transformer procesa una secuencia de tokens $\mathbf{x} = (x_1, \dots, x_n)$ a través de L capas. Cada capa l consta de:

- Una subcapa de **atención multi-cabeza** (MHA) que escribe en el **residual stream** $\mathbf{h}^{(l)}$.
- Una subcapa **feed-forward** (FFN) que también escribe en el residual stream.

El residual stream es el canal de comunicación central del modelo [3]. En cada capa l :

$$\mathbf{h}^{(l+1)} = \mathbf{h}^{(l)} + \text{Attn}^{(l)}(\mathbf{h}^{(l)}) + \text{FFN}^{(l)}(\mathbf{h}^{(l)})$$

donde $\mathbf{h}^{(0)}$ es la suma de la embedding de token y la embedding posicional. La predicción final se obtiene aplicando *layer normalization* seguida de la matriz de *unembedding* $\mathbf{W}_U \in \mathbb{R}^{d \times |\mathcal{V}|}$ al vector $\mathbf{h}^{(L)}$:

$$\text{logits} = \text{LayerNorm}(\mathbf{h}^{(L)}) \cdot \mathbf{W}_U$$

Esta arquitectura es el punto de partida común para todas las técnicas que exploraremos.

Capítulo 2

Logit Lens

2.1. Problema que resuelve

Imaginemos que estamos observando un modelo de lenguaje generar texto token a token. Sabemos qué token predice al final de todo el proceso, pero no tenemos idea de cómo evoluciona esa predicción a lo largo de las capas. ¿El modelo ya sabía la respuesta en capas tempranas y solo la refina? ¿O construye la predicción gradualmente, añadiendo información en cada capa?

La **Logit Lens** [4] es una técnica tan simple como poderosa que responde a esta pregunta: consiste en aplicar directamente la matriz de *unembedding* $\mathbf{W}_U$ a los estados ocultos intermedios del residual stream, como si cada capa fuera la última. Esto nos permite visualizar la distribución de probabilidad sobre el vocabulario que el modelo “tendría” en cada capa si se detuviera en ese punto.

2.2. Fundamento matemático

2.2.1. La idea central

Dado un transformer con L capas, el residual stream en la capa l es $\mathbf{h}^{(l)}$. La predicción final se obtiene como:

$$p_{\text{final}}(x) = \text{softmax}(\text{LayerNorm}(\mathbf{h}^{(L)}) \cdot \mathbf{W}_U)$$

La Logit Lens propone aplicar la misma transformación a cada capa intermedia:

$$\hat{p}^{(l)}(x) = \text{softmax}(\text{LayerNorm}(\mathbf{h}^{(l)}) \cdot \mathbf{W}_U)$$

Esta operación no requiere entrenamiento adicional: reutiliza los parámetros existentes del modelo.

2.2.2. Interpretación como refinamiento iterativo

La hipótesis clave detrás de la Logit Lens es que el residual stream puede interpretarse como un “conducto” que transporta la predicción en evolución del modelo. Cada capa lee el estado actual,

computa una actualización (atención + FFN), y la suma al residual. Desde esta perspectiva, la predicción final emerge como un proceso de refinamiento iterativo:

$$\text{Predicción inicial} \xrightarrow{\text{capa 1}} \text{refinamiento} \xrightarrow{\text{capa 2}} \dots \xrightarrow{\text{capa L}} \text{predicción final}$$

Nostalgebraist (2020) [4] demostró que, en GPT-2 Small, las capas medias ya contienen información semántica sustancial: la distribución $\hat{p}^{(l)}$ para $l \approx L/2$ ya asigna probabilidad no trivial al token correcto, aunque con mucho ruido.

2.2.3. Ejemplo concreto

Consideremos GPT-2 Small procesando la frase “The capital of France is”. La siguiente tabla muestra los principales tokens predichos por Logit Lens en distintas capas:

Capa	Top-1	Top-2	Top-3
0 (embedding)	The	the	capital
2	capital	City	Paris
4	Paris	paris	capital
6	Paris	paris	France
8	Paris	paris	the
10	Paris	paris	France
12 (final)	Paris	paris	France

Observamos que ya en la capa 4 el modelo “sabe” que la respuesta probable es **Paris**, y las capas posteriores simplemente consolidan y refinan esta predicción.

2.3. Variante: Tuned Lens

La Logit Lens original tiene limitaciones importantes (que discutiremos en la Sección 2.4). Para abordarlas, Belrose et al. [5] propusieron la **Tuned Lens**: en lugar de aplicar directamente $\mathbf{W}_U$, se entrena una transformación afín *por capa* antes de la proyección.

2.3.1. Definición formal

Para cada capa l , se define un *translator* $(\mathbf{A}_l, \mathbf{b}_l)$ con $\mathbf{A}_l \in \mathbb{R}^{d \times d}$, $\mathbf{b}_l \in \mathbb{R}^d$. La Tuned Lens se expresa como:

$$\text{TunedLens}_l(\mathbf{h}^{(l)}) = \text{LogitLens}(\mathbf{A}_l \cdot \mathbf{h}^{(l)} + \mathbf{b}_l) = \text{LayerNorm}[\mathbf{A}_l \cdot \mathbf{h}^{(l)} + \mathbf{b}_l] \cdot \mathbf{W}_U$$

2.3.2. Entrenamiento

Los parámetros $(\mathbf{A}_l, \mathbf{b}_l)$ se entrenan para minimizar la divergencia KL entre la distribución de la Tuned Lens y la distribución final del modelo:

$$\arg \min_{\mathbf{A}_l, \mathbf{b}_l} \mathbb{E}_{x \sim \mathcal{D}} \left[D_{\text{KL}}(p_{\text{final}}(\cdot \mid x_{<t}) \parallel \text{TunedLens}_l(\mathbf{h}_t^{(l)})) \right]$$

La función de pérdida usa la distribución final como etiqueta *soft*, lo que evita que el probe aprenda información que el modelo no posee realmente. El entrenamiento requiere aproximadamente 1 hora en 8×A40 para un modelo de 20 mil millones de parámetros.

2.3.3. Mejora cualitativa

Los resultados de la Tuned Lens son dramáticamente mejores:

Métrica	Logit Lens	Tuned Lens
Sesgo (KL divergence)	~4–5 bits	~0.1 bits
Perplejidad en capas tempranas	Muy alta	Baja
Robustez entre arquitecturas	Solo GPT-2 (post-LN)	Cualquier modelo
Parámetros adicionales	0	$\mathbf{A}_l, \mathbf{b}_l$ por capa

2.4. Limitaciones

1. **Fragilidad arquitectónica:** la Logit Lens original funciona bien en GPT-2 (que usa post-layer normalization) pero falla en modelos como GPT-Neo, BLOOM u OPT (pre-layer normalization). La razón es que en estos modelos la distribución del residual stream cambia drásticamente entre capas, haciendo que $\mathbf{W}_U$ sea una mala elección para capas tempranas.
2. **Sesgo sistemático:** la distribución marginal de las predicciones intermedias difiere significativamente de la distribución final. Esto puede inducir a interpretaciones erróneas.
3. **Dimensiones “rogue”:** ciertas dimensiones del residual stream tienen varianza extremadamente alta y distorsionan el producto con $\mathbf{W}_U$, generando picos espurios en la distribución.
4. **Deriva de covarianza:** la covarianza entre el residual stream y $\mathbf{W}_U$ cambia progresivamente a través de las capas, haciendo que la proyección lineal sea subóptima para capas tempranas.
5. **Sin información causal:** la Logit Lens muestra correlaciones entre estados intermedios y predicciones, pero no demuestra que esos estados sean causalmente necesarios para la predicción final.

Capítulo 3

Probing

3.1. Problema que resuelve

La Logit Lens nos dice qué token “cree” el modelo en cada capa. Pero a menudo queremos preguntas más específicas: ¿el modelo sabe si el texto está en francés? ¿Reconoce código Python? ¿Distingue entre singular y plural en la posición del sujeto?

El **probing** (o *análisis con sondas*) [6] es una técnica clásica que entrena clasificadores ligeros—típicamente lineales—sobre las activaciones intermedias del modelo para detectar si cierta información está codificada en ellas. Si un clasificador lineal puede predecir una propiedad y a partir de las activaciones $\mathbf{a}^{(l)}$ con alta precisión, concluimos que esa propiedad está *representada* (linealmente) en esa capa.

3.2. Fundamento matemático

3.2.1. Definición formal

Sea $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ un conjunto de datos donde $\mathbf{x}_i$ es un texto e y_i es una propiedad de interés (binaria o multiclase). Para una capa l , extraemos las activaciones $\mathbf{a}_i^{(l)} \in \mathbb{R}^d$ correspondientes a cada ejemplo. Entrenamos un probe lineal:

$$\hat{y} = \sigma(\mathbf{w}_l^\top \cdot \mathbf{a}^{(l)} + b_l)$$

donde σ es la función sigmoide (para clasificación binaria) o softmax (multiclase), y $(\mathbf{w}_l, b_l)$ son los parámetros del probe para la capa l .

3.2.2. Métrica de “representación”

La precisión del probe en *test*—no en entrenamiento—se interpreta como una medida de cuán accesible es la propiedad y desde las activaciones de la capa l . Si la precisión es significativamente mayor que aleatoria, concluimos que la información está presente.

Definición 3.1 (Selectividad). Un probe es *selectivo* si utiliza únicamente la información que el modelo realmente codifica, sin aprender propiedades espurias que el modelo no usa [7]. Esta propiedad es crítica: un probe demasiado complejo (no lineal, con muchos parámetros) puede aprender a extraer información que no está realmente “disponible” en la representación.

3.2.3. Hipótesis de linealidad

Una premisa fundamental del probing lineal es que las representaciones internas de los transformers organizan la información semántica de forma que sea linealmente accesible. Esto está respaldado por evidencia empírica [8]: las direcciones en el espacio de representaciones codifican relaciones semánticas que pueden recuperarse con operaciones vectoriales lineales.

3.3. Sparse Probing

El probing clásico identifica *capas* relevantes, pero no *neuronas individuales*. El **sparse probing** [9] añade una restricción de esparcidad para localizar features a nivel de neurona.

3.3.1. Definición

Dado un probe lineal denso $\mathbf{w} \in \mathbb{R}^d$, el sparse probing busca una versión k -sparse:

$$\min_{\mathbf{w}, b} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \sigma(\mathbf{w}^\top \cdot \mathbf{a}_i^{(l)} + b)) \quad \text{sujeto a: } \|\mathbf{w}\|_0 \leq k$$

donde $\|\mathbf{w}\|_0$ es la norma ℓ_0 (número de coeficientes no nulos).

3.3.2. Métodos de solución

El problema ℓ_0 es NP-duro. En la práctica se usan aproximaciones:

1. **Diferencia de medias:** ordenar neuronas por $|\mu_{\text{pos}} - \mu_{\text{neg}}|/\sigma$, tomar top- k .
2. **Información mutua:** $I(\text{neurona}; \text{etiqueta})$.
3. **Coefficientes ℓ_1 :** entrenar probe con regularización L1, tomar top- k por magnitud.
4. **Umbral adaptativo:** reducir k iterativamente mientras la precisión se mantiene.
5. **OSP (Optimal Sparse Probing):** algoritmo de cutting-plane con optimalidad demostrable para k pequeños.

3.3.3. Hallazgos empíricos

Gurnee et al. [9] aplicaron sparse probing a 7 modelos (70M a 6.9B parámetros) con más de 100 features lingüísticas y encontraron:

- **Superposición masiva en capas tempranas:** las primeras capas FFN (25%) responden a colecciones de n-gramas no relacionados, evidencia de que las neuronas son polisemánticas.
- **Neuronas dedicadas en capas medias:** aproximadamente al 40–60% de profundidad aparecen neuronas monosemánticas para features como `is_python_code`, `is_french`.
- **Patrones de escalamiento:** modelos más grandes tienden a tener representaciones más esparsas, pero con dinámicas variadas (emergencia de nuevas neuronas, división, comportamiento aleatorio).

3.4. Ejemplo práctico

Supongamos que queremos saber si un modelo BERT distingue entre sujeto singular y plural. Creamos un conjunto de datos con frases como:

Frase	Etiqueta
The dog runs fast	singular
The dogs run fast	plural
The cat sleeps	singular
The cats sleep	plural

Extraemos las activaciones de la posición del sujeto en cada capa y entrenamos un probe lineal. Si el probe alcanza $> 90\%$ de precisión en la capa 8 pero solo 55% en la capa 2, concluimos que la información de número gramatical emerge o se vuelve accesible alrededor de la capa 8.

3.5. Limitaciones

1. **Correlación $\neq$ causalidad:** el probe puede detectar información correlacionada con y que el modelo no utiliza realmente. Hewitt y Liang [7] demostraron con el *control task* que probes demasiado complejos sobreajustan a correlaciones espurias.
2. **Ambigüedad interpretativa:** encontrar que una neurona correlaciona con “francés” no significa que esa neurona sea causal para las predicciones en francés.
3. **Granularidad arbitraria:** no existe una definición universal de “feature”. Diferentes granularidades producen diferentes resultados.
4. **Solo features binarias (en sparse probing):** la mayoría de los estudios se centran en clasificación binaria.
5. **Ilusiones de interpretabilidad:** inspeccionar los máximos activadores de una neurona puede sugerir patrones que no se sostienen bajo análisis riguroso [10].

Capítulo 4

Sparse Autoencoders (SAE)

4.1. Problema que resuelve

Una de las observaciones más perturbadoras de la Mecanistic Interpretabilidad es el fenómeno de **superposición** (superposition) [11]: los modelos representan *más features que dimensiones en sus activaciones*. Esto significa que una neurona individual raramente codifica una sola característica (monosemanticidad); en su lugar, las neuronas son **polisemánticas**, respondiendo a combinaciones de múltiples features.

Los **Sparse Autoencoders** (SAE) [12, 13] son una tecnología diseñada precisamente para desenredar esta superposición. La idea es entrenar un autoencoder con una capa oculta sobredimensionada y una activación escasa (típicamente ReLU o TopK) que descompone las activaciones densas del modelo en una combinación lineal de features monosemánticos.

4.2. Fundamento matemático

4.2.1. El problema de la superposición

Consideremos un espacio de activaciones $\mathbb{R}^d$ donde el modelo representa $m \gg d$ features. La teoría de *toy models* [11] muestra que, cuando las features son escasas (cada feature aparece en una fracción pequeña de los inputs), el modelo puede “comprimir” más de d features en d dimensiones mediante combinaciones lineales casi ortogonales.

La superposición implica que las interpretaciones a nivel de neurona individual son inherentemente limitadas: cada neurona responde a una mezcla de features, y cada feature se distribuye entre varias neuronas.

4.2.2. Arquitectura del SAE

Un SAE es un autoencoder con una sola capa oculta que tiene más dimensiones que la entrada ($F \gg d$):

$$\text{Encoder: } \mathbf{f}(\mathbf{x}) = \text{ReLU}(\mathbf{W}_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_{\text{enc}}) \in \mathbb{R}^F$$

$$\text{Decoder: } \hat{\mathbf{x}} = \mathbf{W}_{\text{dec}} \cdot \mathbf{f}(\mathbf{x}) + \mathbf{b}_{\text{pre}} \in \mathbb{R}^d$$

donde $\mathbf{x} \in \mathbb{R}^d$ son las activaciones del modelo, $\mathbf{f} \in \mathbb{R}^F$ son las activaciones de los features latentes (con $F \gg d$, típicamente $F = 8d$ a $64d$). La matriz $\mathbf{W}_{\text{dec}}$ tiene sus columnas normalizadas a norma unitaria para evitar degeneraciones.

4.2.3. Función de pérdida

La función de pérdida combina fidelidad de reconstrucción con esparcidad:

$$\mathcal{L} = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2}_{\text{pérdida de reconstrucción}} + \alpha \underbrace{\|\mathbf{f}(\mathbf{x})\|_1}_{\text{penalización L1}}$$

El término L1 fuerza a que la mayoría de los features latentes estén inactivos (cerca de cero) para cualquier input dado, mientras que el término de reconstrucción asegura que los features activos capturen información suficiente para reconstruir $\mathbf{x}$.

4.2.4. Variante: TopK SAE

Una alternativa a la penalización L1 es el SAE con activación TopK [14]: en lugar de ReLU, se usa una activación que solo mantiene los k valores más grandes:

$$\mathbf{f}(\mathbf{x}) = \text{TopK}(\mathbf{W}_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_{\text{enc}}, k)$$

Esto fuerza directamente que exactamente k features estén activos por input, eliminando la necesidad de ajustar α .

4.2.5. Entrenamiento

El entrenamiento sigue estos pasos:

1. Recolectar un conjunto grande de activaciones del modelo objetivo (típicamente 10^7 – 10^9 tokens) pasando texto por el modelo y capturando las activaciones en una capa específica.
2. Inicializar el SAE con pesos aleatorios y normalizar las columnas de $\mathbf{W}_{\text{dec}}$.
3. Entrenar con Adam, típicamente con learning rate 10^{-3} a 10^{-4} .
4. **Resampling**: periódicamente (cada 1000 pasos), reemplazar features latentes que nunca se activan (muertos) con nuevas direcciones aleatorias.
5. Para el SAE con L1, ajustar α para lograr una esparcidad objetivo (típicamente ~ 1 – 10% de features activos por input).

4.2.6. Features muertas

Un problema persistente es que una fracción sustancial de los features latentes nunca se activan (hasta 65% en entrenamientos grandes). Esto representa capacidad desperdiciada. Las técnicas para mitigarlo incluyen el resampling periódico, la inicialización data-driven, y normalización dinámica de gradientes.

4.3. Ejemplo práctico

Entrenamos un SAE con $d = 768$ (como la dimensión de GPT-2 Small) y $F = 16 \times 768 = 12\,288$ features latentes. Tras entrenar con 10^8 tokens, inspeccionamos los features que se activan:

ID Feature	Tipos de texto que activan	Interpretación
47	“...in Python...”, “...in Java...”	Código de programación
312	“She said...”, “He exclaimed...”	Verbos de habla
891	“...with $\pm$...”, “...approximately...”	Aproximación numérica

Cada feature es notablemente más monosemántico que las neuronas individuales del modelo base.

4.4. Limitaciones

1. **Costo computacional:** entrenar SAEs requiere cantidades ingentes de datos y recursos. El estudio de *Scaling Monosemanticity* [15] usó 4 millones de horas de TPU para entrenar SAEs en Claude 3 Sonnet.
2. **Features muertas:** capacidad desperdiciada que requiere resampling.
3. **Hiperparámetros sensibles:** α (L1), k (TopK), F/d requieren ajuste cuidadoso.
4. **Interpretabilidad imperfecta:** no todos los features latentes son interpretables para los humanos. Muchos permanecen opacos incluso tras inspección manual.
5. **Superposición entre capas:** los features del SAE capturan features en una capa concreta, pero los features pueden estar distribuidos entre múltiples capas.
6. **Ambigüedad fundamental:** los features del SAE son soluciones a un problema de optimización. No hay garantía de que sean los “verdaderos” features del modelo [16].

Capítulo 5

Automated Feature Explanation

5.1. Problema que resuelve

Una vez que hemos entrenado un SAE y obtenido cientos o miles de features latentes, necesitamos entender qué representa cada uno. La inspección manual es inviable a escala: ¿cómo etiquetar 12 288 features?

La **Automated Feature Explanation** [17], propuesta por OpenAI en 2023, ofrece una solución: usar un LLM (en su caso GPT-4) para generar y evaluar automáticamente explicaciones en lenguaje natural de cada feature.

5.2. El pipeline de tres fases

5.2.1. Fase 1: Generación de explicación

Para cada feature f , se recopilan los 10 textos del conjunto de validación que más activan ese feature. Dado que cada feature puede activarse en múltiples contextos, se toman los k textos con mayor $f(\mathbf{x})$.

Estos 10 textos se presentan a GPT-4 con el siguiente prompt estructurado:

```
1 Estamos estudiando una neurona en un modelo de lenguaje.
2 Estos son los textos que m s activan esta neurona:
3 [texto fragmentado 1]
4 [texto fragmentado 2]
5 ...
6 [texto fragmentado 10]
7 Qu tienen en com n estos textos? Explica
8 brevemente qu detecta esta neurona.
```

GPT-4 produce una explicación como “detectar verbos en pasado” o “identificar referencias a código Python”.

5.2.2. Fase 2: Simulación

La misma explicación se usa para predecir activaciones en textos nuevos. Se presenta a GPT-4:

```
1 Explicaci n de la neurona: "detecta_verbos_en_pasado"
2 Se activar a esta neurona en el siguiente texto?
```

```

3 Texto: "Yesterday, she walked to the store"
4 Respuesta (S /No):

```

GPT-4 responde “Sí” o “No” para cada texto de un conjunto de validación.

5.2.3. Fase 3: Puntuación

Se compara la predicción de GPT-4 con la activación real del feature. Si la activación real está en el top- k de activaciones del batch, se considera un “Sí”; en caso contrario, “No”. La puntuación es:

$$\text{Score} = \max(0, \text{Precisión}_{\text{GPT}} - \text{Precisión}_{\text{mayoritaria}})$$

donde $\text{Precisión}_{\text{mayoritaria}}$ es una línea base (siempre predecir la clase más frecuente). Una puntuación significativamente positiva indica que la explicación captura información genuina sobre el comportamiento del feature.

5.3. Resultados empíricos

Bills et al. [17] aplicaron este pipeline a GPT-2 y GPT-4 (como modelo fuente y como modelo explicador), obteniendo:

Modelo fuente	Features explicables	Score medio	Correlación con humanos
GPT-2 Small (MLP)	95 %	0.35	0.70
GPT-4 (Early SAEs)	69 %	0.28	0.65

Es importante notar que “explicable” no significa que la explicación sea completa o causalmente precisa, solo que captura algún aspecto sistemático del comportamiento del feature.

5.4. Ejemplo práctico

Consideremos un feature que se activa fuertemente en estos textos:

- “...no one knows how the story actually ended...”
- “...they eventually admitted the truth...”
- “...she finally found out what happened...”

GPT-4 genera la explicación: “Detecta adverbios de tiempo que indican resolución o descubrimiento (actually, eventually, finally).”

En textos de validación nuevos:

- “He finally understood the problem” → predice: Sí, activación real: Sí.
- “The sky is blue today” → predice: No, activación real: No.

Score obtenido: 0.42 (significativo).

5.5. Limitaciones

1. **Dependencia del LLM explicador:** la calidad de las explicaciones depende críticamente de la capacidad del LLM. GPT-4 es caro de ejecutar; modelos más pequeños dan explicaciones muy inferiores.
2. **Simulación imperfecta:** predecir activaciones a partir de una explicación textual es una tarea difícil, incluso para LLMs avanzados.
3. **Falsos positivos:** una explicación puede obtener buena puntuación por razones espurias (ej., porque el LLM usa atajos lingüísticos en lugar de comprender realmente el feature).
4. **Explicaciones incompletas:** la explicación generada puede capturar solo un aspecto del feature. Los features polisemánticos reciben explicaciones parciales.
5. **Sin verificación causal:** las explicaciones son correlacionales: describen qué correlaciona con el feature, no qué lo causa ni qué causan las manipulaciones del feature [16].
6. **Cobertura incompleta:** no todos los features son explicables. Los features con activación baja o ruidosa son difíciles de caracterizar.

Capítulo 6

Mean Ablation

6.1. Problema que resuelve

Una vez que sospechamos que un componente del modelo (una cabeza de atención, una neurona FFN, una capa completa) es importante para una tarea, necesitamos una forma de *verificar* experimentalmente esa importancia. ¿Qué le pasa al modelo si eliminamos ese componente?

La **Mean Ablation** [18] es una técnica de intervención que reemplaza la activación de un componente por su valor medio calculado sobre un conjunto de referencia. Si el modelo deja de funcionar correctamente tras la ablación, concluimos que el componente es causalmente relevante.

6.2. Fundamento matemático

6.2.1. Definición

Sea $f(\mathbf{x})$ la activación de un componente (ej., la salida de una cabeza de atención) cuando el modelo procesa el input $\mathbf{x}$. Definimos el conjunto de referencia $\mathcal{D}_{\text{ref}}$ como un conjunto de textos donde el comportamiento del modelo es típico.

La activación ablacionada es:

$$f_{\text{abl}}(\mathbf{x}) = \mathbb{E}_{\mathbf{x}' \sim \mathcal{D}_{\text{ref}}} [f(\mathbf{x}')]]$$

es decir, la media empírica sobre $\mathcal{D}_{\text{ref}}$.

6.2.2. Propiedades

La Mean Ablation tiene propiedades específicas que la distinguen de otras formas de ablación:

- **Preserva la escala:** la activación reemplazada tiene una magnitud típica (la media), a diferencia de ponerla a cero (zero-ablation).
- **Elimina la información específica del input:** al reemplazar $f(\mathbf{x})$ por un valor constante, eliminamos cualquier información que el componente estuviera aportando sobre $\mathbf{x}$.
- **Sesgo hacia activaciones típicas:** si la distribución de $f(\mathbf{x})$ es multimodal, la media puede no ser representativa.

6.2.3. Aplicación: el circuito IOI

Wang et al. [18] usaron Mean Ablation para descubrir el circuito de Identificación de Objeto Indirecto (IOI) en GPT-2 Small. En la tarea IOI, el modelo recibe una frase como:

“Mary and John went to the store. John gave a gift to”

y debe predecir **Mary** (el objeto indirecto, no el sujeto de la segunda cláusula).

Ablacionando cabezas de atención una por una, encontraron que:

- Las cabezas **S-Inhibition** (posiciones 10.7, 11.10) reducían la probabilidad de “Mary” en $\sim 40\%$ cuando se ablacionaban.
- Las cabezas **Duplicate Token** (posiciones 0.1, 0.2) eran responsables de detectar tokens duplicados.
- El ablandamiento del circuito completo requería ablacionar simultáneamente unas 13 cabezas de atención y 2 capas MLP.

6.3. Ejemplo práctico

Supongamos que estamos analizando un modelo que suma dos números de 2 cifras. Queremos saber si la cabeza de atención en la capa 4, posición 7 es importante para acarrear la decena.

1. Calculamos la activación media de esa cabeza sobre 10 000 sumas aleatorias.
2. Cuando el modelo procesa “57 + 38”, intervenimos la cabeza de atención en la capa 4, posición 7, reemplazando su salida por la media.
3. Observamos que la precisión del modelo baja del 95 % al 60 % en sumas con acarreo, pero se mantiene en sumas sin acarreo.
4. Conclusión: esa cabeza es causalmente relevante para el acarreo.

6.4. Limitaciones

1. **Dependencia del conjunto de referencia:** la media calculada depende del conjunto de textos usado. Si $\mathcal{D}_{\text{ref}}$ no es representativo, la ablación puede ser engañosa.
2. **Pérdida de información distribucional:** al reemplazar por un valor constante, ignoramos la varianza natural del componente. Un componente puede ser importante precisamente porque su activación varía entre inputs.
3. **Efectos de segundo orden:** la ablación de un componente puede afectar a otros componentes corriente arriba, produciendo efectos indirectos difíciles de desenredar.
4. **Componentes redundantes:** si varios componentes son funcionalmente equivalentes, ablacionar uno solo no produce efecto apreciable (falso negativo).
5. **No es adecuada para componentes no lineales:** si el componente tiene una función de activación no lineal, reemplazar su salida por la media puede producir efectos inesperados.

Capítulo 7

Path Patching

7.1. Problema que resuelve

La Mean Ablation nos dice si un componente individual es importante. Pero en un transformer, los componentes interactúan formando *rutas computacionales*. Por ejemplo, una cabeza de atención en la capa 5 escribe en el residual stream, que luego es leído por la capa FFN en la capa 6, que a su vez influye en la cabeza de atención en la capa 7. ¿Podemos aislar el efecto de esta ruta específica, independientemente de otras rutas paralelas?

El **Path Patching** [18, 19] responde a esta pregunta. Es una técnica de intervención causal que permite aislar el efecto de una ruta específica entre componentes del modelo, manteniendo el resto del cómputo intacto.

7.2. Fundamento matemático

7.2.1. Configuración experimental

El Path Patching requiere dos ejecuciones del modelo (forward passes):

1. **Forward “clean”**: el modelo procesa el input original $\mathbf{x}_{\text{clean}}$ y produce la predicción normal.
2. **Forward “corrupted”**: el modelo procesa un input alterado $\mathbf{x}_{\text{corrupted}}$ donde se ha cambiado algún aspecto del input (ej., se ha modificado el sujeto de la frase).

Se registran todas las activaciones intermedias de ambos forwards.

7.2.2. El procedimiento de patching

Para aislar la ruta del nodo A al nodo B (donde A y B son componentes como cabezas de atención o capas FFN), realizamos un tercer forward:

1. Partimos del input corrupto $\mathbf{x}_{\text{corrupted}}$.
2. En el punto donde el nodo A produce su salida, forzamos esa activación al valor que tenía en el forward clean (es decir, escribimos A_{clean} en lugar de $A_{\text{corrupted}}$).

3. Esto significa que A recibe input corrupto pero produce la salida que tendría con input limpio.
4. En el nodo B , leemos la activación correspondiente (que ahora será diferente porque su entrada ha cambiado).
5. Medimos cuánto se acerca la predicción final a la del forward clean.

Formalmente, definimos el **Efecto Indirecto** (IE) de la ruta $A \rightarrow B$ como:

$$IE_{A \rightarrow B} = p(\text{clean} \mid \mathbf{x}_{\text{corrupted}} \text{ con } A_{\text{clean}} \text{ parcheado en } B) - p(\text{clean} \mid \mathbf{x}_{\text{corrupted}})$$

7.2.3. Tipos de Path Patching

Existen dos variantes principales:

1. **Activación patching**: actúa sobre las activaciones del componente A (es decir, su salida). Es la variante descrita arriba.
2. **Gradiente patching**: actúa sobre el gradiente de la pérdida respecto a las activaciones de A . Es computacionalmente más eficiente pero menos directo.

7.3. Ejemplo práctico: el circuito IOI

Wang et al. [18] aplicaron Path Patching para confirmar la arquitectura del circuito IOI.

Consideremos la frase:

- $\mathbf{x}_{\text{clean}}$: “Mary and John went to the store. John gave a gift to”
- $\mathbf{x}_{\text{corrupted}}$: “Mary and John went to the store. Alice gave a gift to”

En el forward clean, el modelo predice “Mary”. En el forward corrupto (cambiando John $\rightarrow$ Alice en la segunda cláusula), el modelo predice “Alice” (porque el objeto indirecto en la frase limpia era “Mary” pero ahora es “Alice”... en realidad predeciría “Mary” porque Mary sigue siendo el indirect object en la frase original... veamos con cuidado).

En la tarea IOI, la estructura es: “[A] and [B] went to the store. [B] gave a gift to” $\rightarrow$ predecir [A].

Si corrompemos a: “[A] and [B] went to the store. [C] gave a gift to” $\rightarrow$ el modelo predecirá algo diferente.

Path Patching permite preguntar: “¿la ruta desde la cabeza **Duplicate Token** (que detecta que [B] aparece dos veces) hasta la cabeza **S-Inhibition** (que suprime la predicción de [B]) es causalmente relevante?”

La respuesta experimental es sí: parcheando esta ruta se restaura la predicción correcta incluso con input corrupto.

7.4. Limitaciones

1. **Costo computacional:** cada Path Patching requiere $\mathcal{O}(L^2)$ forwards en el peor caso, donde L es el número de componentes.
2. **Linealidad asumida:** el Path Patching asume que los efectos de las rutas son aproximadamente lineales y aditivos, lo que no siempre es cierto en transformers reales.
3. **Elección de puntos de intervención:** la elección de A y B requiere hipótesis previas. No es un método de descubrimiento sino de verificación.
4. **Interacciones entre rutas:** las rutas pueden tener efectos no aditivos, haciendo que intervenciones independientes no revelen el panorama completo.
5. **Definición de “ruta”:** en transformers, prácticamente todo el cómputo confluye en el residual stream, lo que hace difícil aislar rutas verdaderamente independientes.

Capítulo 8

Random Ablation

8.1. Problema que resuelve

La Mean Ablation reemplaza una activación por su media. Pero la media puede no ser representativa de la distribución real de activaciones. Si la activación de un componente tiene alta varianza, reemplazarla por un valor constante introduce un artefacto que puede distorsionar el comportamiento del modelo.

La **Random Ablation** [20] ofrece una alternativa: en lugar de usar un valor constante, se reemplaza la activación por la activación del *mismo componente* en otro input aleatorio del conjunto de referencia. Esto preserva las propiedades distribucionales de la activación (media, varianza y correlaciones con otros componentes), eliminando solo el acoplamiento causal entre el input y esa activación.

8.2. Fundamento matemático

8.2.1. Definición

Sea $f(\mathbf{x})$ la activación de un componente para input $\mathbf{x}$. Elegimos un input aleatorio $\mathbf{x}_{\text{ref}} \sim \mathcal{D}_{\text{ref}}$ y definimos:

$$f_{\text{rand}}(\mathbf{x}) = f(\mathbf{x}_{\text{ref}})$$

La clave es que $\mathbf{x}_{\text{ref}}$ se elige independientemente de $\mathbf{x}$ y de forma aleatoria.

8.2.2. Propiedades deseables

La Random Ablation tiene propiedades estadísticas superiores a la Mean Ablation:

- **Preserva la varianza:** la activación reemplazada tiene la misma varianza que la original.
- **Preserva correlaciones:** si f está correlacionada con otros componentes g , la correlación se mantiene (aunque ahora es espuria), evitando artefactos.
- **Distribución realista:** cada muestra de ablación procede de una activación que realmente ocurrió, no de un valor sintético.

8.2.3. Promedio sobre múltiples ablaciones

Para obtener una estimación robusta, se repite la Random Ablation N veces con diferentes inputs aleatorios y se promedia el efecto:

$$\overline{\text{Efecto}} = \frac{1}{N} \sum_{i=1}^N [p(\text{clean} \mid \mathbf{x} \text{ con } f_{\text{rand}_i}) - p(\text{clean} \mid \mathbf{x})]$$

8.3. Comparación: Mean Ablation vs. Random Ablation

Propiedad	Mean Ablation	Random Ablation
Valor de reemplazo	$\mathbb{E}[f]$	$f(\mathbf{x}_{\text{ref}})$
Preserva media	Sí	En expectativa
Preserva varianza	No	Sí
Preserva correlaciones	No	Sí
Costo (por ablación)	Bajo (1 forward)	Medio (N forwards)
Adecuado para componentes con alta varianza	No	Sí

8.4. Ejemplo práctico

Para ilustrar la diferencia, consideremos un componente cuya activación tiene una distribución bimodal: $f(\mathbf{x}) \in \{-1, 1\}$ con igual probabilidad. La media es 0, pero el componente solo tiene efecto cuando su activación es ± 1 (no cuando es 0, que nunca ocurre naturalmente).

- **Mean Ablation:** reemplaza por 0. El modelo se comporta como si el componente estuviera en un estado que nunca ocurre en la práctica. Esto puede producir efectos artificiales.
- **Random Ablation:** reemplaza por -1 o 1 aleatoriamente. El modelo se comporta como si el componente hubiera recibido un input diferente, pero con activaciones realistas.

8.5. Limitaciones

1. **Dependencia del conjunto de referencia:** al igual que Mean Ablation, el conjunto $\mathcal{D}_{\text{ref}}$ debe ser representativo.
2. **Varianza de la estimación:** para obtener estimaciones precisas, se necesitan múltiples muestras aleatorias, aumentando el costo computacional.
3. **Correlaciones espurias mantenidas:** aunque preserva correlaciones de la distribución original, la correlación con el input específico se pierde. Si dos componentes covarían con el input pero son causalmente independientes, Random Ablation puede preservar artefactos.
4. **No es adecuada para componentes pequeños:** si la varianza del componente es muy pequeña, Random Ablation se aproxima a Mean Ablation pero con ruido adicional.

Capítulo 9

Causal Scrubbing

9.1. Problema que resuelve

Las técnicas de ablación que hemos visto hasta ahora verifican hipótesis sobre componentes individuales o rutas. Pero las hipótesis interpretativas completas—“el circuito C implementa el comportamiento B ”—son más complejas: involucran múltiples componentes, interacciones y condiciones.

El **Causal Scrubbing** [20, 21] es un framework formal para verificar hipótesis interpretativas completas. Proporciona un protocolo para determinar si una hipótesis es causalmente fiel al modelo, y una métrica cuantitativa de “qué parte del comportamiento explica la hipótesis”.

9.2. Fundamento matemático

9.2.1. La hipótesis como grafo

Una hipótesis interpretativa H se representa como un *grafo causal* sobre las activaciones del modelo. Los nodos del grafo son variables (activaciones de componentes) y las aristas representan dependencias causales entre ellas.

Formalmente, H especifica:

1. Una partición de cada activación en subcomponentes (ej., descomponer la salida de una cabeza de atención en contribuciones de cada token de entrada).
2. Las dependencias causales entre esos subcomponentes: qué subcomponentes influyen en qué otros.
3. Un conjunto de *condiciones de resampling*: para cada subcomponente, cuándo se puede reemplazar por el valor de otro input.

9.2.2. El procedimiento de scrubbing

El Causal Scrubbing se ejecuta como sigue:

1. Partimos del input original $\mathbf{x}_{\text{clean}}$.

2. Seleccionamos un conjunto de inputs de referencia $\mathcal{D}_{\text{ref}}$.
3. Para cada subcomponente s en H :
 - a) Si s es independiente de la predicción según H , su activación se reemplaza por la de un input aleatorio $\mathbf{x}_{\text{ref}}$.
 - b) Si s depende de algún otro subcomponente s' , se usa el input que fue seleccionado para s' (resampling condicional).
4. Medimos la métrica de interés (ej., log-probabilidad de la respuesta correcta) en el modelo “scrubbed”.

9.2.3. Resampling condicional

La clave de Causal Scrubbing es el **resampling condicional**. En lugar de reemplazar activaciones con valores fijos o aleatorios independientes, se usa un procedimiento recursivo:

Algoritmo 9.1 (Resampling condicional). Para cada subcomponente s :

1. Identificar los subcomponentes ancestros de s según H .
2. Para cada ancestro a , obtener el input $\mathbf{x}_a$ que se usó para resamplear a .
3. Si todos los ancestros usan el mismo input $\mathbf{x}'$, entonces resamplear s usando $\mathbf{x}'$.
4. Si los ancestros usan diferentes inputs, hay un conflicto: la hipótesis H es incompleta o incorrecta para este ejemplo.

Si el modelo scrubbed preserva la predicción original, la hipótesis es causalmente fiel. Si el modelo falla, la hipótesis es incorrecta o incompleta.

9.3. Métrica: *Recovered Log-Probability*

La métrica principal de Causal Scrubbing es la *recovered log-probability*:

$$\text{RLP}(H) = \frac{\log p_{\text{scrubbed}}(y \mid x) - \log p_{\text{baseline}}(y \mid x)}{\log p_{\text{clean}}(y \mid x) - \log p_{\text{baseline}}(y \mid x)}$$

donde p_{baseline} es la probabilidad del token correcto bajo la distribución marginal (sin información del input). $\text{RLP} = 1$ significa que la hipótesis explica todo el comportamiento; $\text{RLP} = 0$ significa que no explica nada.

9.4. Ejemplo práctico

Consideremos una hipótesis simple sobre el circuito IOI: “La predicción de **Mary** depende únicamente de la cabeza **Duplicate Token L0H0** y la cabeza **S-Inhibition L10H7**”.

1. Ejecutamos Causal Scrubbing con esta hipótesis sobre la frase “Mary and John went to the store. John gave a gift to”.

2. El resampling condicional reemplaza todas las demás activaciones por valores de inputs aleatorios.
3. Si obtenemos $RLP > 0,9$, la hipótesis es esencialmente correcta: esas dos cabezas son las únicas relevantes.
4. Si obtenemos $RLP < 0,3$, la hipótesis es insuficiente: hay otros componentes importantes que no hemos considerado.

9.5. Limitaciones

1. **Complejidad exponencial:** el árbol de resampling puede crecer exponencialmente con el número de subcomponentes.
2. **Formulación de hipótesis costosa:** especificar una hipótesis completa en el formato de Causal Scrubbing es laborioso.
3. **Conflictos de resampling:** es común encontrar conflictos donde los ancestros usan diferentes inputs, lo que indica que la hipótesis necesita refinamiento.
4. **Hipótesis atómicas:** el framework asume que los subcomponentes son independientes y sus efectos aditivos, lo que puede no ser cierto.
5. **Cobertura:** incluso con RLP alto, la hipótesis puede estar perdiendo detalles finos que solo se manifiestan en ejemplos concretos.

Capítulo 10

Activation Patching

10.1. Problema que resuelve

Hasta ahora, las técnicas de intervención que hemos visto (Mean Ablation, Path Patching) requieren una hipótesis específica sobre qué componente es importante. Pero a menudo queremos *descubrir* qué componentes son relevantes para una tarea sin hipótesis previas—una tarea de localización.

El **Activation Patching** (también llamado *Causal Tracing* en la literatura) [22] es una técnica de localización causal que sistemáticamente interviene cada componente del modelo para medir su contribución individual a la predicción.

10.2. Fundamento matemático

10.2.1. Configuración experimental

Al igual que en Path Patching, el Activation Patching requiere dos estados del modelo:

1. **Forward clean:** $\mathbf{x}_{\text{clean}}$ produce la predicción p_{clean} .
2. **Forward corrupted:** $\mathbf{x}_{\text{corrupted}}$ produce $p_{\text{corrupted}}$.

La diferencia crucial es que ahora intervenimos *cada activación por separado*, midiendo el **Efecto Indirecto Total** (TIE) de cada una.

10.2.2. Procedimiento paso a paso

Para una activación específica $a_{l,c}^{(t)}$ (en capa l , componente c , posición de token t):

1. Ejecutamos el forward con input corrupto.
2. En el momento en que se calcula $a_{l,c}^{(t)}$, reemplazamos su valor por el que tenía en el forward clean.
3. Continuamos el forward hasta la salida.

4. Medimos cómo cambia la probabilidad del token correcto.

Este procedimiento se repite para cada activación de interés, requiriendo $\mathcal{O}(L \times C \times T)$ forwards, donde L es el número de capas, C el número de componentes por capa y T la longitud de la secuencia.

10.2.3. Efecto Indirecto Total

El TIE de la activación a se define como:

$$\text{TIE}(a) = p_{\text{clean}} \mid_{\text{do}(a=a_{\text{clean}}), \text{input}=x_{\text{corrupted}}} - p_{\text{corrupted}}$$

Un TIE alto indica que la activación es causalmente importante: restaurarla en un contexto corrupto recupera la predicción original.

10.3. Aplicación: ROME (Model Editor)

Meng et al. [22] usaron Activation Patching para localizar las activaciones responsables de almacenar hechos factuales en GPT-2 XL. Descubrieron que:

- Las capas FFN medias (capas 11–17 en GPT-2 XL de 48 capas) son las que almacenan información factual.
- Una sola capa FFN (típicamente alrededor del 30–40 % de la profundidad del modelo) es responsable del hecho.
- Activando esta capa FFN en modo clean mientras el resto del modelo recibe input corrupto, se recupera la predicción correcta.

Basándose en este hallazgo, desarrollaron **ROME** (Rank-One Model Editing): un método para editar hechos factuales modificando directamente los pesos de la capa FFN localizada.

10.4. Ejemplo práctico

Queremos saber qué capas almacenan el hecho “La capital de España es Madrid”.

1. $\mathbf{x}_{\text{clean}}$: “La capital de España es” $\rightarrow$ modelo predice “Madrid” ($p \approx 0,85$).
2. $\mathbf{x}_{\text{corrupted}}$: “La capital de Francia es” $\rightarrow$ modelo predice “París” ($p \approx 0,90$).
3. Para cada capa l , intervenimos las activaciones FFN:
 - Capa 8: TIE $\approx 0,01$ (sin efecto).
 - Capa 12: TIE $\approx 0,65$ (¡efecto grande!).
 - Capa 16: TIE $\approx 0,02$.
4. Conclusión: la capa 12 es donde se almacena el hecho “España $\rightarrow$ Madrid”.

10.5. Limitaciones

1. **Costo computacional prohibitivo:** para un modelo con $L = 96$ capas (como PaLM), $C = 4$ componentes (atención + FFN) y $T = 1024$ tokens, se necesitan $\sim 400\,000$ forwards para cubrir todas las activaciones.
2. **Intervenciones independientes:** el Activation Patching analiza cada activación por separado, ignorando interacciones entre componentes.
3. **Linealidad:** el TIE asume que los efectos son aproximadamente lineales, lo que puede no ser cierto para transformaciones no lineales como softmax.
4. **Elección de input corrupto:** el TIE depende críticamente de cómo se corrompa el input. Diferentes corrupciones pueden producir diferentes mapas de importancia.
5. **Redundancia:** si varios componentes son funcionalmente equivalentes, el TIE individual de cada uno puede ser bajo porque el resto compensa.

Capítulo 11

ACDC (Automatic Circuit DisCovery)

11.1. Problema que resuelve

El Activation Patching y el Path Patching requieren intervenir cada componente manualmente y analizar los resultados. Para modelos con miles de componentes, esto es inviable manualmente.

ACDC (Automatic Circuit DisCovery) [23] automatiza este proceso. Dado un modelo y una tarea, ACDC descubre automáticamente el subgrafo de componentes (el *circuito*) que es causalmente relevante para la tarea.

11.2. Fundamento matemático

11.2.1. El algoritmo

ACDC es un algoritmo recursivo que itera desde la salida del modelo hacia la entrada. El pseudocódigo es:

Algoritmo 11.1 (ACDC).

```
1 def acdc(model, input_clean, input_corrupt, threshold):
2     # 1. Ejecutar forwards clean y corrupto
3     clean_cache = model.run_with_cache(input_clean)
4     corrupt_cache = model.run_with_cache(input_corrupt)
5
6     # 2. Definir el grafo computacional completo
7     full_graph = build_computation_graph(model)
8
9     # 3. Descubrimiento recursivo
10    def recursive_prune(node, graph):
11        for child in node.children:
12            # Mide el efecto de aislar child
13            effect = measure_effect(child, graph)
14            if effect < threshold:
15                graph.remove_edge(node, child)
16            else:
17                recursive_prune(child, graph)
18
19    # 4. Empezar desde el nodo de salida
20    output_node = full_graph.get_output_node()
21    recursive_prune(output_node, full_graph)
22    return graph
```

11.2.2. Métrica de efecto

Para decidir si una arista (o nodo) debe mantenerse, ACDC usa una métrica basada en divergencia KL entre la distribución del modelo completo y la del subgrafo:

$$\text{Efecto}(e) = D_{\text{KL}}(p_{\text{completo}} \parallel p_{\text{subgrafo} \setminus e})$$

Si eliminar la arista e no aumenta significativamente la divergencia KL (por debajo de un umbral τ), la arista se elimina permanentemente.

11.2.3. El umbral τ

El umbral τ controla el equilibrio entre *exhaustividad* (encontrar todas las aristas relevantes, pero con falsos positivos) y *minimalidad* (encontrar solo las esenciales, pero con riesgo de falsos negativos). Conmy et al. [23] proponen una búsqueda binaria sobre τ para maximizar:

$$\mathcal{F}_1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

donde la precisión y el recall se definen respecto al circuito de referencia descubierto manualmente.

11.3. Resultados empíricos

Conmy et al. aplicaron ACDC a GPT-2 Small en la tarea *Greater-Than* (determinar si un número es mayor que otro):

- El circuito descubierto manualmente tenía ~ 68 aristas.
- ACDC redescubrió 5 de 5 tipos de componentes del circuito manual.
- Precisión: 89 %, Recall: 92 % respecto al ground truth manual.
- El circuito completo del modelo tiene $\sim 32\,000$ aristas. ACDC las redujo a ~ 68 (reducción de $470\times$).

11.4. Ejemplo práctico

Para la tarea “¿57 > 38?”, el grafo computacional completo de GPT-2 Small tiene 32 000 aristas. ACDC lo reduce a:

Componente	Rol en el circuito
Cabeza atención 3.0	Mueve el operando izquierdo a la posición de resultado
Cabeza atención 5.5	Mueve el operando derecho a la posición de resultado
FFN capa 7	Compara los valores numéricos
FFN capa 9	Produce la decisión binaria

El resto de las 32 000 aristas son irrelevantes para esta tarea específica (aunque pueden ser importantes para otras tareas).

11.5. Limitaciones

1. **Costo cuadrático en el peor caso:** ACDC requiere $\mathcal{O}(N^2)$ forwards en el peor caso, donde N es el número de aristas.
2. **Dependencia del umbral:** la elección de τ afecta drásticamente los resultados. No hay una guía universal para elegirlo.
3. **Específico de una tarea:** el circuito descubierto es válido solo para la tarea y el conjunto de datos usado.
4. **Falsos negativos con componentes redundantes:** si dos componentes tienen la misma función, ACDC puede eliminar uno, que en realidad sí contribuye en otros contextos.
5. **No captura comportamientos emergentes:** el algoritmo asume una relación jerárquica entre componentes, que puede no reflejar comportamientos emergentes complejos.

Capítulo 12

Attribution Patching

12.1. Problema que resuelve

El Activation Patching es el estándar de oro para medir la importancia causal de cada activación, pero su costo $\mathcal{O}(L \times C \times T)$ lo hace prohibitivo para modelos grandes. Por ejemplo, para Pythia-12B (40 capas, 20 000 puntos de activación), Activation Patching requeriría semanas de cómputo.

El **Attribution Patching** (AtP) [24] aborda este problema usando una aproximación de Taylor de primer orden. En lugar de intervenir cada activación individualmente, AtP estima el efecto de *todas* las intervenciones simultáneamente con solo 3 pases (2 forwards + 1 backward).

12.2. Fundamento matemático

12.2.1. La aproximación de Taylor

Sea $L(\mathbf{x})$ la función de pérdida del modelo y $\mathbf{e}$ el vector de todas las activaciones del modelo. Queremos medir el cambio en la pérdida cuando cambiamos las activaciones de su valor en el forward clean $\mathbf{e}_{\text{clean}}$ a su valor en el forward corrupto $\mathbf{e}_{\text{corrupt}}$:

$$\Delta L = L(\mathbf{e}_{\text{corrupt}}) - L(\mathbf{e}_{\text{clean}})$$

AtP aproxima esto con el desarrollo de Taylor de primer orden alrededor de $\mathbf{e}_{\text{clean}}$:

$$\Delta L \approx \nabla_{\mathbf{e}} L(\mathbf{e}_{\text{clean}})^\top \cdot (\mathbf{e}_{\text{corrupt}} - \mathbf{e}_{\text{clean}})$$

De esta forma, la contribución de cada activación individual i es:

$$\text{AtP}_i = \left. \frac{\partial L}{\partial e_i} \right|_{\mathbf{e}_{\text{clean}}} \cdot (e_i^{(\text{corrupt})} - e_i^{(\text{clean})})$$

12.2.2. Cómputo eficiente

AtP requiere solo:

1. **Forward clean:** guardar $\mathbf{e}_{\text{clean}}$ y calcular $L(\mathbf{e}_{\text{clean}})$.

2. **Backward clean:** calcular $\nabla_{\mathbf{e}} L(\mathbf{e}_{\text{clean}})$.
3. **Forward corrupt:** guardar $\mathbf{e}_{\text{corrupt}}$.
4. **Producto elemento a elemento:** $\text{AtP}_i = (\nabla_{\mathbf{e}} L)_i \cdot \Delta e_i$.

Todo esto se realiza en $\mathcal{O}(1)$ pases, independientemente del número de activaciones.

12.2.3. Interpretación intuitiva

Podemos entender AtP así: el gradiente $\partial L / \partial e_i$ nos dice qué tan sensible es la pérdida a cambios en la activación e_i . Si multiplicamos esa sensibilidad por cuánto cambió realmente la activación (Δe_i), obtenemos una estimación de cuánto contribuye e_i al cambio total en la pérdida.

12.3. Fallos conocidos (y AtP*)

Kramár et al. [26] identificaron dos modos de fallo sistemáticos en AtP:

12.3.1. Fallo por saturación de softmax

Si la activación clean ya produce una probabilidad cercana a 1 (softmax saturado), el gradiente es cercano a cero y AtP subestima la importancia de esa activación.

Solución (QK-fix): recalcular explícitamente el softmax con queries y keys intervenidas, en lugar de usar el gradiente estándar.

12.3.2. Fallo por cancelación frágil

Si dos activaciones tienen efectos opuestos que se cancelan, AtP puede mostrar un efecto neto pequeño (sugiriendo que no son importantes), cuando en realidad ambas son críticas.

Solución (Dropout en backward): añadir dropout al backward pass para romper correlaciones espurias entre gradientes.

12.4. Ejemplo práctico

Queremos identificar las 10 activaciones más importantes para la tarea “La capital de Francia es París” vs. “La capital de Alemania es Berlín”.

1. Forward clean: “La capital de Francia es” $\rightarrow$ pérdida L_{clean} .
2. Backward: calcular $\nabla_{\mathbf{e}} L$ para todas las activaciones.
3. Forward corrupt: “La capital de Alemania es” $\rightarrow$ guardar $\mathbf{e}_{\text{corrupt}}$.
4. Calcular AtP_i para todas las activaciones.
5. Ordenar por $|\text{AtP}_i|$ y seleccionar top-10.

Resultado: las activaciones más importantes están en la FFN de la capa 12 y las cabezas de atención que procesan “Francia” y “París”. AtP identificó estas en segundos, mientras que Activation Patching habría requerido minutos u horas.

12.5. Limitaciones

1. **Aproximación lineal:** la precisión de AtP depende de cuán lineal sea el modelo en la región de interés. Los transformers tienen muchas no-linealidades (ReLU, softmax, atención), por lo que la aproximación puede ser pobre.
2. **Falsos negativos:** los fallos de saturación y cancelación pueden ocultar componentes importantes (AtP* los aborda parcialmente).
3. **Sin interacciones entre activaciones:** AtP ignora términos cruzados. Si dos activaciones interactúan de forma no lineal, AtP no las captura.
4. **Elección de la tarea:** la importancia de las activaciones depende críticamente de la elección del par clean/corrupt.
5. **No es un sustituto del patching real:** AtP es una aproximación. Para confirmación causal definitiva, se necesita Activation Patching.

Capítulo 13

Edge Attribution Patching (EAP)

13.1. Problema que resuelve

El Activation Patching (incluyendo ACDC) analiza la importancia de activaciones individuales (nodos en el grafo computacional). Sin embargo, en un transformer, la información fluye a través de *aristas* entre componentes: la contribución de una cabeza de atención pasa por una matriz de proyección de valor (OV circuit) antes de escribir en el residual stream. Si queremos encontrar el *circuito mínimo*—el subgrafo más pequeño que mantiene el comportamiento—necesitamos medir la importancia de cada arista, no solo de cada nodo.

El **Edge Attribution Patching** (EAP) [25] aplica la idea de Attribution Patching a cada arista del grafo computacional, permitiendo identificar y seleccionar las k aristas más importantes para una tarea.

13.2. Fundamento matemático

13.2.1. De nodos a aristas

En el grafo computacional de un transformer, una arista representa el flujo de información entre dos componentes. Por ejemplo:

- Arista `embeddings` $\rightarrow$ `MLP_layer_5`: la contribución de la embedding a la capa FFN 5.
- Arista `head_3.1` $\rightarrow$ `MLP_layer_8`: la contribución de la cabeza de atención en capa 3, posición 1, a la capa FFN 8.

EAP estima la importancia de cada arista usando la misma aproximación de Taylor que AtP:

$$\text{EAP}(e_{i \rightarrow j}) = \left. \frac{\partial L}{\partial e_{i \rightarrow j}} \right|_{\text{clean}} \cdot (e_{i \rightarrow j}^{(\text{corrupt})} - e_{i \rightarrow j}^{(\text{clean})})$$

13.2.2. El algoritmo EAP

Algoritmo 13.1 (EAP). 1. Ejecutar forward clean y guardar $\mathbf{e}_{\text{clean}}$ (todas las aristas).

2. Ejecutar forward corrupt y guardar $\mathbf{e}_{\text{corrupt}}$.
3. Ejecutar backward desde la pérdida clean para obtener $\nabla_{\mathbf{e}} L$.
4. Calcular EAP_i para cada arista i :

$$\text{EAP}_i = (\nabla_{\mathbf{e}} L)_i \cdot \Delta e_i$$

5. Ordenar aristas por $|\text{EAP}_i|$ (importancia).
6. Seleccionar las top- k aristas como el circuito candidato.

13.2.3. Comparación EAP vs. ACDC

La ventaja fundamental de EAP sobre ACDC es la eficiencia:

Métrica	ACDC	EAP
Pases forward	$\mathcal{O}(N^2)$	2
Pases backward	0	1
Calidad (AUC en IOI)	0.868	0.904
Correlación con patching real (R^2)	1.0	0.27
Escalabilidad a modelos grandes	Mala	Excelente

EAP obtiene mejor AUC que ACDC porque considera la importancia de cada arista de forma continua (en lugar de la decisión binaria de ACDC), pero su correlación con el patching real es modesta ($R^2 = 0.27$).

13.3. Variante: EAP con integradores

Para mejorar la precisión de EAP, Syed et al. [25] propusieron usar *integradores*: en lugar de una sola pareja clean/corrupt, se usan múltiples parejas y se promedian los scores. Esto reduce la varianza y mejora la correlación con el patching real.

$$\text{EAP-IG}(e) = \frac{1}{M} \sum_{m=1}^M (\nabla_{\mathbf{e}} L^{(m)})_e \cdot (e_{\text{corrupt}}^{(m)} - e_{\text{clean}}^{(m)})_e$$

13.4. Ejemplo práctico

Para la tarea IOI en GPT-2 Small, EAP identifica las aristas más importantes:

Arista (origen $\rightarrow$ destino)	Función	Score EAP
emb $\rightarrow$ head 10.7	Detectar nombre duplicado	0.42
head 10.7 $\rightarrow$ head 11.10	Suprimir sujeto repetido	0.38
head 0.1 $\rightarrow$ head 10.7	Copiar atención previa	0.31
head 11.10 $\rightarrow$ MLP 11	Computar predicción final	0.29

El circuito resultante mantiene $> 95\%$ del rendimiento original con solo $< 1\%$ de las aristas totales.

13.5. Limitaciones

1. **Baja correlación con el ground truth:** aunque EAP identifica las aristas correctas en el ranking (alto AUC), la magnitud de los scores individuales no se correlaciona bien con el efecto real del patching ($R^2 \approx 0,27$).
2. **Dependencia del input:** los resultados de EAP varían significativamente según la pareja clean/corrupt elegida.
3. **Solo aristas directas:** EAP considera aristas individuales, no interacciones entre aristas ni caminos de largo >1 .
4. **Aproximación lineal:** al igual que AtP, la calidad de EAP depende de la linealidad local del modelo.
5. **No hay umbral claro:** la decisión de cuántas aristas mantener (top- k) es arbitraria y depende de la tarea.

Capítulo 14

Conclusiones

En este informe hemos explorado en detalle doce técnicas fundamentales de Mechanistic Interpretability para modelos de lenguaje basados en transformers. Cada técnica aborda una pregunta específica sobre el funcionamiento interno del modelo:

1. **Logit Lens** responde: ¿qué token predice el modelo en cada capa intermedia?
2. **Probing** responde: ¿esta información específica está representada en esta capa?
3. **SAE** responde: ¿podemos descomponer las activaciones en features monosemánticas?
4. **Automated Feature Explanation** responde: ¿qué significado tiene cada feature latente?
5. **Mean Ablation** responde: ¿es este componente causalmente importante?
6. **Path Patching** responde: ¿esta ruta específica entre componentes es causal?
7. **Random Ablation** responde: ¿es importante la varianza de este componente?
8. **Causal Scrubbing** responde: ¿es esta hipótesis completa causalmente fiel?
9. **Activation Patching** responde: ¿qué activaciones son causalmente relevantes para esta tarea?
10. **ACDC** responde: ¿podemos descubrir el circuito completo automáticamente?
11. **Attribution Patching** responde: ¿cómo aproximar eficientemente el patching?
12. **Edge Attribution Patching** responde: ¿qué aristas del grafo computacional son importantes?

14.1. Tabla comparativa

Técnica	Tipo	Costo	Cobertura	Precisión causal
Logit Lens	Correlacional	$\mathcal{O}(1)$	Global (token)	Ninguna
Probing	Correlacional	$\mathcal{O}(L)$	Local (feature)	Ninguna
SAE	Descomposición	Muy alto	Local (feature)	Ninguna
Auto. Feature Expl.	Descriptivo	Alto	Local (feature)	Ninguna
Mean Ablation	Intervención	$\mathcal{O}(N)$	Local (componente)	Moderada
Path Patching	Intervención	$\mathcal{O}(N^2)$	Local (ruta)	Alta
Random Ablation	Intervención	$\mathcal{O}(N \times M)$	Local (componente)	Alta
Causal Scrubbing	Verificación	Muy alto	Global (hipótesis)	Muy alta
Activation Patching	Intervención	$\mathcal{O}(L \times C \times T)$	Local (activación)	Muy alta
ACDC	Descubrimiento	$\mathcal{O}(N^2)$	Global (circuito)	Alta
Attribution Patching	Aproximación	$\mathcal{O}(1)$	Global (activaciones)	Moderada
Edge Attribution Patching	Aproximación	$\mathcal{O}(1)$	Global (aristas)	Moderada

14.2. Recomendaciones prácticas

Para investigadores que se inician en Mecanistic Interpretability, recomendamos el siguiente flujo de trabajo:

1. **Exploración inicial:** usar **Logit Lens** para visualizar la evolución de las predicciones a través de las capas.
2. **Formulación de hipótesis:** usar **Probing** y **SAE** para identificar features y componentes que correlacionan con el comportamiento de interés.
3. **Etiquetado:** usar **Automated Feature Explanation** para caracterizar los features descubiertos.
4. **Localización rápida:** usar **Attribution Patching** o **EAP** para identificar rápidamente las activaciones o aristas candidatas.
5. **Confirmación causal:** usar **Activation Patching** y **Path Patching** para verificar que los candidatos son causalmente relevantes.
6. **Descubrimiento sistemático:** usar **ACDC** para encontrar el circuito completo.
7. **Verificación formal:** usar **Causal Scrubbing** para validar que la hipótesis del circuito es completa y fiel.

14.3. Direcciones futuras

La Mecanistic Interpretability es un campo en rápida evolución. Algunas direcciones abiertas incluyen:

- **Escalamiento:** ¿cómo aplicar estas técnicas a modelos con cientos de miles de millones de parámetros?
- **Automatización:** ¿podemos automatizar completamente el pipeline de descubrimiento de circuitos?
- **Universalidad:** ¿los circuitos descubiertos en un modelo se transfieren a otros modelos y arquitecturas?
- **Intervención:** ¿podemos usar el conocimiento de los circuitos para mejorar el rendimiento, la robustez o la seguridad de los modelos?
- **Integración con entrenamiento:** ¿podemos diseñar arquitecturas y procedimientos de entrenamiento que produzcan modelos inherentemente más interpretables?

Estas preguntas marcan la frontera de la investigación en Mecanistic Interpretability, y las técnicas presentadas en este informe constituyen las herramientas fundamentales para abordarlas.

Bibliografía

- [1] A. Vaswani et al., “Attention is all you need,” en *Advances in Neural Information Processing Systems*, 2017.
- [2] A. Radford et al., “Language models are unsupervised multitask learners,” OpenAI Technical Report, 2019.
- [3] N. Elhage et al., “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [4] Nostalgebraist, “Interpreting GPT: the logit lens,” *LessWrong*, 2020. [Online]. Available: <https://www.lesswrong.com/posts/AcKRB8wDpdAN6v6ru/interpreting-gpt-the-logit-lens>
- [5] N. Belrose et al., “Eliciting latent predictions from transformers with the tuned lens,” *arXiv:2303.08112*, 2023.
- [6] G. Alain y Y. Bengio, “Understanding intermediate layers using linear classifier probes,” en *ICLR Workshop*, 2017.
- [7] J. Hewitt y P. Liang, “Designing and interpreting probes with control tasks,” en *EMNLP*, 2019.
- [8] T. Mikolov et al., “Distributed representations of words and phrases and their compositionality,” en *NIPS*, 2013.
- [9] W. Gurnee et al., “Finding neurons in a haystack: case studies with sparse probing,” *TMLR*, 2023.
- [10] T. Bolukbasi et al., “Interpretability and dataset artifacts: a cautionary tale,” en *ACL Workshop*, 2021.
- [11] N. Elhage et al., “Toy models of superposition,” *Transformer Circuits Thread*, 2022.
- [12] T. Bricken et al., “Towards monosemanticity: decomposing language models with dictionary learning,” *Transformer Circuits Thread*, 2023.
- [13] H. Cunningham et al., “Sparse autoencoders find highly interpretable features in language models,” en *ICLR*, 2024.
- [14] L. Gao et al., “Scaling and evaluating sparse autoencoders,” *arXiv:2406.04093*, 2024.
- [15] A. Templeton et al., “Scaling monosemanticity: extracting interpretable features from Claude 3 Sonnet,” *Anthropic*, 2024.

- [16] A. Mäkelov et al., “A framework for identifying interpretable directions in neural representations,” en *ICLR*, 2024.
- [17] S. Bills et al., “Language models can explain neurons in language models,” *OpenAI*, 2023.
- [18] K. Wang et al., “Interpretability in the wild: a circuit for indirect object identification in GPT-2 small,” en *ICLR*, 2023.
- [19] N. Goldowsky-Dill et al., “Localizing model behavior with path patching,” *arXiv:2304.05969*, 2023.
- [20] Redwood Research, “Causal scrubbing: a method for rigorously testing interpretability hypotheses,” *LessWrong*, 2022.
- [21] A. Geiger et al., “Causal abstraction: a theoretical foundation for mechanistic interpretability,” *arXiv:2301.04709*, 2023.
- [22] K. Meng et al., “Locating and editing factual associations in GPT,” en *NeurIPS*, 2022.
- [23] A. Conmy et al., “Towards automated circuit discovery for mechanistic interpretability,” en *NeurIPS*, 2023.
- [24] N. Nanda, “Attribution patching: activation patching at industrial scale,” 2022. [Online]. Available: <https://www.neelnanda.io/mechanistic-interpretability/attribution-patching>
- [25] A. Syed et al., “Attribution patching outperforms automated circuit discovery,” en *NeurIPS ATTRIB Workshop*, 2023.
- [26] J. Kramár et al., “AtP*: an efficient and scalable method for localizing LLM behavior to components,” *arXiv:2403.00745*, 2024.